

Complete Traceability & Observability Guide for Agentic RAG Systems

This document provides a complete implementation guide for adding Traceability and Observability to an Agentic RAG system using 6 major tools.

1. Azure AI Foundry Observability

- Native Azure observability solution integrated with Application Insights and Azure Monitor.
- Captures traces, spans, latency, token usage, tool calls, and agent decisions.
- Best for Azure-hosted Agentic RAG deployments.

Example Integration Code

```
from langchain_azure_ai.callbacks.tracingv2 import AzureAIInferenceTracer

tracer = AzureAIInferenceTracer(
    connection_string=os.environ[
        "APPLICATION_INSIGHTS_CONNECTION_STRING"
    ]
)

response = chain.invoke(
    query,
    config={"callbacks": [tracer]}
)
```

2. Langfuse

- Open-source observability platform for LLM applications.
- Provides excellent dashboards for traces, spans, metrics, and token tracking.
- Very suitable for multi-agent RAG systems.

Example Integration Code

```
from langfuse import Langfuse

langfuse = Langfuse()

trace = langfuse.trace(
    name="agentic_rag_pipeline"
)

span = trace.span(name="intent_agent")

response = client.chat.completions.create(...)

span.end(output=response.model_dump())
```

3. LangSmith

- Observability platform built by the LangChain team.
- Requires almost zero code changes.

- Automatically traces chains, tools, prompts, and agents.

Example Integration Code

```
# .env variables
LANGCHAIN_TRACING_V2=true
LANGCHAIN_API_KEY=your_key

from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4o")

response = llm.invoke("Explain observability")
```

4. Arize Phoenix

- Local-first observability tool optimized for RAG systems.
- Provides groundedness and hallucination analysis.
- Runs completely locally.

Example Integration Code

```
import phoenix as px

px.launch_app()

from openinference.instrumentation.openai import OpenAIInstrumentor

OpenAIInstrumentor().instrument()
```

5. Helicone

- Proxy-based observability system with extremely fast setup.
- Excellent for cost and latency monitoring.
- Limited span-level visibility.

Example Integration Code

```
client = AzureOpenAI(
    azure_endpoint="https://oai.hconeai.com",
    default_headers={
        "Helicone-Auth": "Bearer YOUR_API_KEY"
    }
)
```

6. W&B Weave

- Experiment tracking and tracing platform.
- Decorator-based tracing makes integration easy.
- Useful for ML-heavy pipelines.

Example Integration Code

```
import weave

weave.init("agentic-rag-capstone")
```

Complete Traceability & Observability Guide for Agentic RAG Systems

Introduction

This document provides a complete implementation guide for adding Traceability and Observability to an Agentic RAG (Retrieval-Augmented Generation) system.

The document is based on the following 6 observability tools:

1. Azure AI Foundry Observability
2. Langfuse
3. LangSmith
4. Arize Phoenix
5. Helicone
6. W&B Weave

Each section contains:

- What the tool is
- Why it is useful
- Advantages and disadvantages
- Installation commands
- Integration steps
- Fully commented code
- Multiple implementation examples
- Graph generation commands
- Trace and span visualization guidance
- Metrics captured
- How to integrate into existing code
- Viva explanations

PART 1 — Understanding Traceability & Observability

What is Observability?

Observability means monitoring and understanding everything happening inside an AI system.

In an Agentic RAG pipeline, observability helps answer:

- Which agent is slow?
- Which agent generated incorrect output?
- How many tokens were consumed?
- What was the latency?
- Which agent hallucinated?
- Which query type performs poorly?

Observability tracks:

Signal Meaning

Traces Complete journey of one query

Spans Individual agent operations

Metrics Numerical measurements

Logs Timestamped system events

What is Traceability?

Traceability means following a user query step-by-step through all agents.

Example:

User Query

↓

Intent Agent

↓

Retrieval Agent

↓

Reasoning Agent

↓

Validation Agent

One complete journey = TRACE

Each individual step = SPAN

Important Metrics in Agentic RAG

Metric	Meaning
Latency	Time taken
Tokens	Prompt + completion tokens
Cost	Estimated API usage cost
Faithfulness	Is answer grounded in context?
Relevance	Is answer related to query?
Completeness	Is answer complete?
Confidence	Validation confidence
Accuracy	Correctness of classification

Standard Graphs Used in Observability

Graph	Purpose
Trace Timeline	Shows end-to-end execution
Span Timeline	Shows agent-by-agent timings

Graph	Purpose
Latency Histogram	Distribution of execution times
Token Usage Bar Chart	Tokens used per run
Radar Chart	Compare quality metrics
Pie Chart	Query intent distribution
Line Chart	Score trends across runs
Heatmap	Agent bottlenecks

TOOL 1 — Azure AI Foundry Observability

What is Azure AI Foundry Observability?

Azure AI Foundry Observability is Microsoft Azure's built-in monitoring system for AI applications.

It integrates with:

- Azure OpenAI
- LangChain
- Azure Monitor
- Application Insights
- OpenTelemetry

It automatically records:

- Traces
- Spans
- Latency
- Tool calls
- Token usage
- Model responses

- Agent decisions

Advantages

Advantage	Explanation
Native Azure integration	Works directly with Azure services
OpenTelemetry support	Industry-standard tracing
Production monitoring	Suitable for enterprise systems
Automatic tracing	Minimal code changes
Application Insights	Centralized monitoring

Installation

```
pip install langchain-azure-ai
pip install azure-monitor-opentelemetry
```

Environment Variables

```
# .env file
APPLICATION_INSIGHTS_CONNECTION_STRING=your_connection_string
AZURE_OPENAI_ENDPOINT=your_endpoint
AZURE_OPENAI_API_KEY=your_api_key
```

Basic Azure Observability Integration

```
# =====
# IMPORTS
# =====
```

```

import os

from dotenv import load_dotenv

# Loads variables from .env
load_dotenv()

# Azure tracing callback
from langchain_azure_ai.callbacks.tracingv2 import AzureAIInferenceTracer

# =====

# CREATE TRACER

# =====

# This tracer automatically captures traces
# and spans from LangChain operations
tracer = AzureAIInferenceTracer(
    connection_string=os.environ[
        "APPLICATION_INSIGHTS_CONNECTION_STRING"
    ]
)

# =====

# EXAMPLE LANGCHAIN CHAIN

# =====

from langchain_openai import AzureChatOpenAI

```



```

llm = AzureChatOpenAI(
    azure_endpoint=os.environ["AZURE_OPENAI_ENDPOINT"],
    api_key=os.environ["AZURE_OPENAI_API_KEY"],
    deployment_name="gpt-4o"
)

# =====
# RUN QUERY WITH TRACING
# =====

response = llm.invoke(
    "Explain observability in AI systems",
    config={
        "callbacks": [tracer] # Enables tracing
    }
)

print(response.content)

```

Multiple Agent Example

```

# =====
# AGENT 1 — INTENT CLASSIFICATION
# =====

intent_response = llm.invoke(

```

```
"Classify this query: What was Q1 revenue?",
config={"callbacks": [tracer]}
)
```

```
# =====
# AGENT 2 — RETRIEVAL
# =====
```

```
retrieval_response = llm.invoke(
    "Retrieve financial information",
    config={"callbacks": [tracer]}
)
```

```
# =====
# AGENT 3 — REASONING
# =====
```

```
reasoning_response = llm.invoke(
    "Generate final answer",
    config={"callbacks": [tracer]}
)
```

How to View Traces

Azure Portal

→ AI Foundry

→ Observability

→ Traces

Commands to Generate Metrics Graphs

Azure Monitor automatically generates:

1. Latency Graphs

2. Token Usage Graphs

3. Request Timeline

4. Span Duration Graphs

5. Error Rate Charts

6. Throughput Charts

Important Metrics Captured

Metric	Captured?
Trace Timeline	Yes
Span Duration	Yes
Token Usage	Yes
Latency	Yes
Agent Calls	Yes
Tool Invocations	Yes
Errors	Yes

How to Incorporate into Existing Code

ORIGINAL CODE

```
response = chain.invoke(query)
```

```
# UPDATED CODE

response = chain.invoke(

    query,

    config={"callbacks": [tracer]}

)
```

TOOL 2 — Langfuse

What is Langfuse?

Langfuse is an open-source LLM observability platform.

It provides:

- Traces
- Spans
- Token tracking
- Latency tracking
- Evaluation metrics
- Quality scores
- Dashboard visualizations

It is one of the most popular tools for Agentic RAG systems.

Advantages

Advantage	Explanation
Open-source	Free and customizable
Best dashboard	Excellent UI

Advantage	Explanation
Span-level visibility	Deep debugging
RAG friendly	Designed for AI pipelines
Cloud + local support	Flexible deployment

Installation

```
pip install langfuse
```

```
pip install openai
```

```
pip install python-dotenv
```

Environment Variables

```
LANGFUSE_PUBLIC_KEY=pk-lf-xxxxxx
```

```
LANGFUSE_SECRET_KEY=sk-lf-xxxxxx
```

```
LANGFUSE_HOST=https://cloud.langfuse.com
```

Full Langfuse Integration

```
# =====
```

```
# IMPORTS
```

```
# =====
```

```
import os
```

```
import time
```

```
import json
```

```
from dotenv import load_dotenv
```

```
load_dotenv()
```

```
# =====
```

```
# LANGFUSE IMPORT
```

```
# =====
```

```
from langfuse import Langfuse
```

```
# Create Langfuse client
```

```
langfuse = Langfuse(  
    public_key=os.environ["LANGFUSE_PUBLIC_KEY"],  
    secret_key=os.environ["LANGFUSE_SECRET_KEY"],  
    host=os.environ["LANGFUSE_HOST"]  
)
```

```
# =====
```

```
# OPENAI IMPORT
```

```
# =====
```

```
from openai import AzureOpenAI
```

```
client = AzureOpenAI(  
    azure_endpoint=os.environ["AZURE_OPENAI_ENDPOINT"],  
    api_key=os.environ["AZURE_OPENAI_API_KEY"],  
    api_version="2024-10-21"  
)
```

DEPLOYMENT = "gpt-4o"

Creating a Trace

```
# =====  
# CREATE TRACE  
# =====
```

```
trace = langfuse.trace(  
    name="agentic_rag_pipeline",  
    input={  
        "query": "What was Q1 revenue?"  
    },  
    tags=["capstone", "rag"]  
)
```

Creating Spans

```
# =====  
# CREATE SPAN  
# =====
```

```
span = trace.span(  
    name="intent_agent",  
    input={  
        "query": "What was Q1 revenue?"  
    }  
)
```

```

start = time.time()

# =====

# LLM CALL

# =====

response = client.chat.completions.create(
    model=DEPLOYMENT,
    messages=[
        {
            "role": "user",
            "content": "Classify query intent"
        }
    ]
)

latency = time.time() - start

# =====

# END SPAN

# =====

span.end(
    output={
        "intent": "tabular_query"
    }
)

```



```
},  
usage={  
  "promptTokens": response.usage.prompt_tokens,  
  "completionTokens": response.usage.completion_tokens,  
  "totalTokens": response.usage.total_tokens  
},  
metadata={  
  "latency_seconds": round(latency, 3)  
}  
)
```

Adding Evaluation Scores

```
# =====  
# QUALITY SCORES  
# =====
```

```
langfuse.score(  
  trace_id=trace.id,  
  name="faithfulness",  
  value=0.91  
)
```

```
langfuse.score(  
  trace_id=trace.id,  
  name="relevance",  
  value=0.88
```

)

```
langfuse.score(  
    trace_id=trace.id,  
    name="completeness",  
    value=0.85  
)
```

Full 4-Agent Example

```
# =====
```

```
# AGENT 1 — INTENT AGENT
```

```
# =====
```

```
intent_span = trace.span(name="intent_agent")
```

```
intent_span.end(output={"intent": "hybrid"})
```

```
# =====
```

```
# AGENT 2 — RETRIEVAL AGENT
```

```
# =====
```

```
retrieval_span = trace.span(name="retrieval_agent")
```

```
retrieval_span.end(output={"documents": 4})
```

```
# =====
```

```
# AGENT 3 — REASONING AGENT
```

```
# =====
```

```
reasoning_span = trace.span(name="reasoning_agent")
reasoning_span.end(output={"answer": "Revenue was $2M"})
```

```
# =====
# AGENT 4 — VALIDATION AGENT
# =====
```

```
validation_span = trace.span(name="validation_agent")
validation_span.end(output={"confidence": 0.92})
```

Generating Graphs Using Matplotlib

```
# =====
# IMPORTS
# =====
```

```
import matplotlib.pyplot as plt
```

```
# Example data
```

```
latencies = [2.1, 2.4, 3.0, 1.9]
```

```
agents = [
```

```
    "Intent",
```

```
    "Retrieval",
```

```
    "Reasoning",
```

```
    "Validation"
```

```
]
```

```
# =====  
  
# LATENCY BAR GRAPH  
  
# =====  
  
plt.bar(agents, latencies)  
plt.xlabel("Agents")  
plt.ylabel("Latency")  
plt.title("Agent Latency Comparison")  
plt.show()
```

Radar Chart Example

```
import numpy as np  
import matplotlib.pyplot as plt  
  
metrics = [  
    "Faithfulness",  
    "Relevance",  
    "Completeness",  
    "Confidence"  
]  
  
scores = [0.91, 0.88, 0.85, 0.89]  
  
angles = np.linspace(0, 2*np.pi, len(metrics), endpoint=False)
```

```
scores = np.concatenate((scores, [scores[0]]))
angles = np.concatenate((angles, [angles[0]]))

fig = plt.figure()
ax = fig.add_subplot(111, polar=True)
ax.plot(angles, scores)
ax.fill(angles, scores, alpha=0.25)

plt.title("Quality Metrics Radar Chart")
plt.show()
```

Commands to Run Langfuse Pipeline

```
python observability_runner.py
python generate_graphs.py
```

Graphs Generated

Graph	Purpose
Latency Bar Chart	Compare agent timings
Token Usage Graph	Cost analysis
Radar Chart	Quality metric comparison
Histogram	Latency distribution
Line Graph	Score trends
Pie Chart	Query types

How to Incorporate into Existing Code

BEFORE

```
response = client.chat.completions.create(...)
```

AFTER

```
trace = langfuse.trace(name="pipeline")
```

```
span = trace.span(name="agent")
```

```
response = client.chat.completions.create(...)
```

```
span.end(output=response)
```

TOOL 3 — LangSmith

What is LangSmith?

LangSmith is an observability platform created by the LangChain team.

It automatically traces:

- Chains
- Agents
- Tools
- Prompt templates
- LLM calls

Best suited for LangChain-based systems.

Advantages

Advantage	Explanation
Zero code changes	Easy integration
LangChain native	Direct compatibility
Good debugging	Strong tracing support
Prompt monitoring	Tracks prompt performance

Installation

```
pip install langsmith
```

Environment Variables

```
LANGCHAIN_TRACING_V2=true
LANGCHAIN_API_KEY=your_key
LANGCHAIN_PROJECT=agentic-rag-capstone
```

Simple Integration

```
# =====
# JUST ENABLE ENV VARIABLES
# =====

# No additional tracing code required
# LangChain automatically traces all calls

from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4o")
```

```
response = llm.invoke(  
    "Explain LangSmith"  
)
```

Multiple Agent Example

```
# =====  
# AGENT 1  
# =====
```

```
intent = llm.invoke("Classify intent")
```

```
# =====  
# AGENT 2  
# =====
```

```
retrieval = llm.invoke("Retrieve documents")
```

```
# =====  
# AGENT 3  
# =====
```

```
reasoning = llm.invoke("Generate final answer")
```

Viewing Traces

smith.langchain.com

→ Project Dashboard

→ Traces

Graphs Automatically Generated

Graph	Available
Trace timelines	Yes
Token graphs	Yes
Latency graphs	Yes
Tool usage graphs	Yes
Prompt comparison	Yes

How to Incorporate into Existing Code

Existing LangChain code works automatically

after setting environment variables

TOOL 4 — Arize Phoenix

What is Arize Phoenix?

Arize Phoenix is an open-source observability tool optimized for RAG systems.

It provides:

- Local tracing
- RAG evaluation
- Groundedness metrics
- Offline monitoring
- Prompt debugging

Advantages

Advantage	Explanation
Fully free	No paid account
Local-first	Runs on localhost
RAG-specific metrics	Strong retrieval evaluation
OpenTelemetry compatible	Industry-standard tracing

Installation

```
pip install arize-phoenix
pip install openinference-instrumentation-openai
```

Basic Setup

```
# =====

# IMPORT PHOENIX

# =====

import phoenix as px

# Launch dashboard locally
px.launch_app()

# =====

# IMPORT INSTRUMENTOR

# =====
```

```
from openinference.instrumentation.openai import OpenAIInstrumentor
```

```
# Automatically traces OpenAI calls
```

```
OpenAIInstrumentor().instrument()
```

Example Query

```
from openai import OpenAI
```

```
client = OpenAI()
```

```
response = client.chat.completions.create(
```

```
    model="gpt-4o",
```

```
    messages=[
```

```
        {
```

```
            "role": "user",
```

```
            "content": "Explain RAG systems"
```

```
        }
```

```
    ]
```

```
)
```

Viewing Dashboard

```
http://localhost:6006
```

Metrics Generated

Metric	Available
--------	-----------

Groundedness	Yes
--------------	-----

Retrieval relevance	Yes
---------------------	-----

Hallucination detection	Yes
-------------------------	-----

Span latency	Yes
--------------	-----

Prompt traces	Yes
---------------	-----

Graphs Supported

Phoenix dashboard automatically provides:

1. Trace Graphs

2. Span Timelines

3. Latency Histograms

4. Groundedness Charts

5. Retrieval Quality Charts

TOOL 5 — Helicone

What is Helicone?

Helicone is a lightweight LLM observability proxy.

It focuses on:

- Cost monitoring
- Latency monitoring
- Request logging

It is very easy to integrate.

Advantages

Advantage	Explanation
Fastest setup	2-minute integration
Simple architecture	Easy to understand
Cost tracking	Excellent billing visibility
API proxy model	Minimal changes

Installation

```
pip install openai
```

Basic Integration

```
from openai import AzureOpenAI
```

```
client = AzureOpenAI(  
  
    # Helicone proxy endpoint  
    azure_endpoint="https://oai.hconeai.com",  
  
    # Authentication header  
    default_headers={  
        "Helicone-Auth": "Bearer YOUR_API_KEY"  
    },  
  
    api_key=os.environ[
```

```
"AZURE_OPENAI_API_KEY"
]
)
```

Example Query

```
response = client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {
            "role": "user",
            "content": "Explain observability"
        }
    ]
)
```

Metrics Captured

Metric	Available
Cost tracking	Yes
Latency	Yes
Token usage	Yes
API requests	Yes
Agent spans	No

Dashboard

<https://www.helicone.ai/dashboard>

TOOL 6 — W&B Weave

What is W&B Weave?

W&B Weave is an observability and experiment tracking platform from Weights & Biases.

It is commonly used by:

- ML engineers
 - Data scientists
 - AI researchers
-

Advantages

Advantage	Explanation
Decorator-based tracing	Very easy integration
Experiment tracking	Strong ML workflows
Good dashboards	Clean visualizations
Model comparison	Excellent for evaluation

Installation

```
pip install weave
```

Basic Integration

```
import weave
```

```
# Initialize project
```

```
weave.init("agentic-rag-capstone")
```

Tracing with Decorators

```
# =====  
# TRACE ENTIRE FUNCTION  
# =====
```

```
@weave.op()
```

```
def reasoning_agent(query, context):
```

```
    # This entire function is automatically traced
```

```
    response = client.chat.completions.create(  
        model="gpt-4o",  
        messages=[  
            {  
                "role": "user",  
                "content": query  
            }  
        ]  
    )
```

```
    return response.choices[0].message.content
```

Example Usage

```
answer = reasoning_agent(  
    "What is observability?",
```


"AI monitoring context"

)

Metrics Captured

Metric	Available
Function traces	Yes
Inputs/outputs	Yes
Latency	Yes
Experiment comparisons	Yes
Agent spans	Partial

Comparison Table

Tool	Setup Time	Free Tier	Agent Tracing	Best For
Azure Foundry	15 min	Yes	Yes	Azure deployments
Langfuse	30 min	Yes	Yes	Multi-agent RAG
LangSmith	5 min	Yes	Yes	LangChain apps
Arize Phoenix	10 min	Yes	Yes	RAG evaluation
Helicone	2 min	Yes	Limited	Quick monitoring
W&B Weave	20 min	Yes	Partial	ML experiments

Recommended Architecture for Your Project

Best Combination

For your capstone project:

Purpose	Recommended Tool
Production monitoring	Azure Foundry
Multi-agent tracing	Langfuse
RAG evaluation	Arize Phoenix
LangChain debugging	LangSmith

Best Viva Answer

We evaluated six observability tools:

Azure AI Foundry, Langfuse, LangSmith,
Arize Phoenix, Helicone, and W&B Weave.

We selected Langfuse as the primary tracing solution because it provided detailed span-level visibility for our multi-agent RAG pipeline.

We also used Azure AI Foundry Observability because our deployment runs on Azure, allowing native integration with Application Insights and Azure Monitor.

Final Conclusion

Observability and traceability are critical for Agentic RAG systems because they provide:

- Visibility into agent behavior
- Performance analysis
- Hallucination detection

- Token and cost monitoring
- Debugging support
- Scientific evaluation metrics

We evaluated Azure AI Foundry, Langfuse, LangSmith, Arize Phoenix, Helicone, and W&B Weave for observability in our Agentic RAG system. We selected Langfuse for deep span-level tracing and Azure Foundry for native Azure integration.

END OF DOCUMENT